

WeatherGenerator Architecture

Christian Lessig

European Centre for Medium-Range Weather Forecasts, Bonn, Germany

christian.lessig@ecmwf.int

May 20, 2025

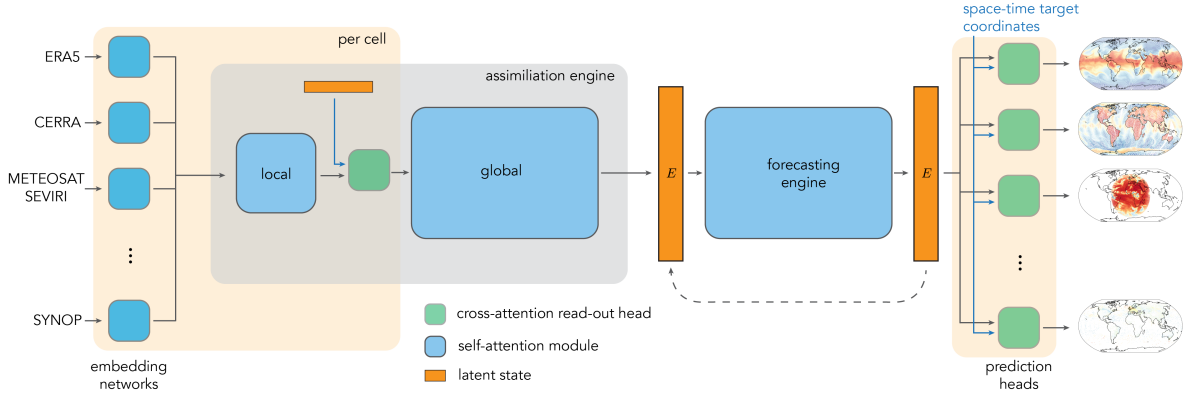


Figure 1: Overview of the WeatherGenerator neural network architecture

1 Nomenclature

E global latent Earth system state

K global latent space dimension ()

R number of latent space vectors per cell ()

S number of latent cells (controlled through latent healpix level)

Δt length of temporal window that is the input/output of the model (currently also time step)

2 Overview

The WeatherGenerator neural network architecture has the overall structure shown in Fig. 1. It is designed to take as input a potentially large number of different but related datasets, both structured and unstructured. The model combines these into a joint latent Earth system state E in the assimilation engine. E subsumes the information from the different inputs and allows to decode information about the system within its temporal support. This is accomplished with output-specific prediction heads.

The latent space E provides a richer and more complete description of the system state than any individual input dataset. After training, it can be instantiated from a sparse set of input datasets with well calibrated uncertainties, i.e. so that decoded quantities provide physically valid samples that have a well calibrated spread consummate with the provided inputs. In the limit case, when only climatological input is provided (e.g. a date), the output are realistic Earth system states whose distribution matches the climatology.

To decouple the latent space from the input datasets, E consists of an abstract set of tokens. These are organized around a subdivision of the sphere, i.e. the latent space tokens are associated with cells

of the subdivision with R latent vectors per cell. One can think of it as a hedgehog with K -dimensional spines and R spines per cell.¹ Since its quasi-uniform and has a well-defined hierarchical structure, the healpix scheme is employed.

The WeatherGenerator will ingest data from a Δt -window from the input datasets. On the one hand, this is necessary to be able to use datasets with different temporal resolutions and samplings, including observations which might be at arbitrary times. On the other hand, the use of a window that represents a system trajectory better constrains the system state and its possible future evolution than an instantaneous state. Output will be at arbitrary times within the Δt -window associated with E .

Remark 1. *The design of the WeatherGenerator is inspired by the bitter lessons of machine learning² that state that with sufficient data and compute, the optimization/training will outperform handcrafted features. The inductive biases that the network has are therefore largely for computational reasons to be able to process the 4-dimensional data in Earth system science.*

3 Input and embedding networks

Inputs are organized into *streams*, each of which can consist of multiple *sources*. Each stream has the same channels and through the multiple sources one can, e.g., combine very similar datasets. Each stream has one embedding network (encoder) and one prediction head (decoder) associated with it. For different streams, the embedding and prediction are fully independent.

Each *input item* of a stream consists of a location, of additional geo-information, and of physical values associated with the location. The physical values will be referred to as *channels*. For ERA5, e.g., one has one location associated with each vertical column and the channels correspond to the different physical fields across the different levels. For satellite observations, the channels are the satellite channels and geo-information is, e.g., the satellites zenith angle. The geo-information can also be positional, e.g. when one has modern radiosonde measurements that track position along the ascent, this can be encoded through the geo-information with the location corresponding to start locations.

The streams are processed per healpix cell for a fixed, chosen healpix level that matches the information density of the dataset (per stream healpix levels are not fully implemented yet). For each input, hence the healpix cell associated with its location is determined and all inputs within a cell are collected. When the number of inputs exceeds a threshold, these are split (although a hierarchical subdivision that follows the healpix scheme until each cell does not exceed the threshold would be more appropriate).

Remark 2. *A hierarchical subdivision of the inputs based on local density is appropriate, e.g., for SYNOP stations whose spatial distribution is highly imbalanced. However, then one would also have one stream being simultaneously associated with multiple scale levels. One can still have one per-stream embedding network but the embedded tokens are associated with different levels.*

The embedding networks are currently transformer-based and operate on the information per local healpix cell per stream. Two approaches are thereby supported at the moment. The first one performs an embedding per input item along the physical values and attention is computed between the items in the pe-cell patch. The second approach computes the embedding across channels for each cell and attention is between the channels. The motivation for this per-channel embedding is that the values for a fixed channel in a local spatial patch have a high regularity/correlations and thus an efficient embedding (e.g. compression) is possible. With a per input item embedding, in contrast, one has jumps in the signal that is embedded (e.g. because the channels for a forecast or reanalysis are for different physical fields or because the channels for an observational input are correlated with different fields).

Remark 3. *With transformer-based embedding networks the number of inputs per cell can be arbitrary. Hence, in principle, no splitting is necessary. Earlier experiments showed, however, that splitting and*

¹Formally, this is a K -dimensional vector bundle over S^2 .

²<http://www.incompleteideas.net/IncIdeas/BitterLesson.html>. In [?] this was recently formulated as “Our work reinforces the bitter lesson. The most important factors determining the performance of a sensibly designed model are the compute and data available for training” where the authors define “sensibly designed” as “models that are sufficiently expressive and have stable gradient propagation”.

padding to have a fixed number outperforms variable length input. These experiments should be repeated. Having an upper bound or a fixed size also enforces an upper bound on the memory requirements.

Remark 4. An alternative to the embedding transformers are graph neural networks. For fully regular meshes, one could also employ simpler approaches, like the linear layers of vision transformers. Experience shows, however, that combining different embedding networks and prediction heads can be challenging, likely because the different gradient flows need careful balancing.

Remark 5. One approach for parallelizing the network for very high resolutions is to follow a classical domain decomposition approach and apply the embedding network for different regions on different compute nodes. Then on each node also only local data needs to be loaded and pre-processed, which otherwise can also easily become a bottleneck. A major challenge is, however, how to efficiently aggregate information for coarser levels with a domain decomposition approach.

4 Projection heads

The purpose of the projection heads is to translate from latent space to physical space. For this, they take the local, per cell latent state E_i (or the k -ring neighborhood) as well as per stream target coordinates as input and produce the physical field values corresponding to the target coordinates. This is implemented using a transformer with the target coordinates as queries and the latent per cell state E_i as key/values pairs, see Sec. 7 below for details.

The motivation for using a local approach is that this likely generalizes better, although further experiments are required. The target coordinates are encoded as local, per cell coordinates, e.g. w.r.t. the cell center and the vertices of the cell.

Remark 6. When the prediction is global and the entire latent space is used to read out an arbitrary value (which is possible and provides skilful predictions) then the latent space becomes asymptotically unstructured (in the depth of the network).

5 Assimilation Engine

The assimilation engine consists of three parts, cf. Fig. 1. It first combines the different input streams in the local assimilation engine, projects the output onto a fixed set of tokens in a read-out head, and then combines the local states in the global assimilation engine to obtain E .

The local assimilation engine blends the embedded tokens (or per cell information) from the different streams for each cell using a regular, dense transformer. It works on tokens that are still associated with the input items and hence the number of tokens depends on the spatial region as well as the input datasets and their data density.

To have a regular and predictable set of tokens for further processing, the output of the local assimilation engine is mapped onto a fixed number R of tokens per cell. These tokens are initialized randomly; experiments show that there is no benefit by using different read-out tokens for different cells (this is somewhat surprising and perhaps worth understanding better). These tokens are used as per-cell query tokens with the per-cell local assimilation engine output as key/value tokens, cf. Sec. 7. Applying this projection for every cell yields $S \times R$ latent vectors. These are processed by the global assimilation engine that is a transformer with sparse/dense self-attention and that combines the local information into a globally consistent state. The sparse attention uses local neighborhoods on S^2 , which can be defined conveniently through the healpix scheme. (The local attention is implemented with torch’s flex attention.)

The output of the global assimilation engine forms the global latent Earth system state E . It is given by $S \times R$ latent vectors organized around the cell of the subdivision.

6 Forecasting Engine

The global system state is associated with a temporal window Δt in which physical values can be decoded. The forecasting engine advances the window by a fixed amount.

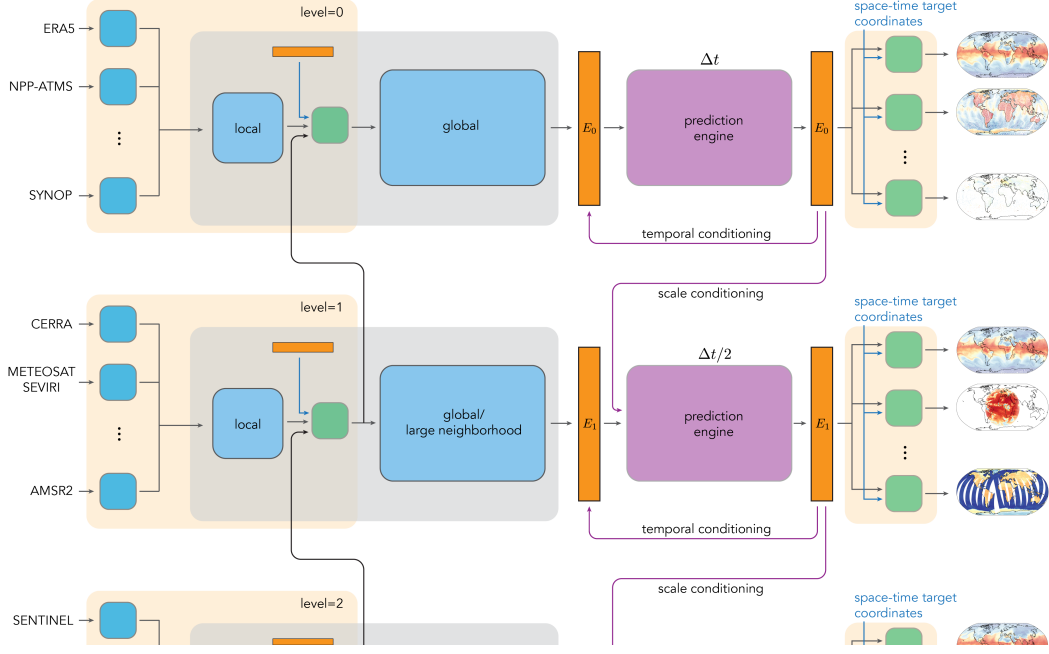


Figure 2: Envisioned multi-resolution structure for the WeatherGenerator. The column of conditional prediction networks is a multi-scale diffusion model similar to [?]. From an Earth science perspective, it implements downscaling. Forecasting with a conditioning on past information and a scale-adapted time step is also realized by the prediction engines F_j that generate .

Remark 7. *Currently the window is advanced by Δt . However, having overlapping window, e.g. a 12h window and a 6h advance is an interesting approach that should be explored.*

The forecasting engine is currently deterministic and implemented by a transformer with sparse/dense self-attention, analogous to the global assimilation engine. It is trained in a second step after the embedding networks, prediction heads and assimilation engine have been pre-trained with an auto-encoder-like approach using masked token modeling or short-term forecasting.

7 Transformer read-out heads

An important aspect of the WeatherGenerator is the ability to map between different representations. For this, currently transformer read-out heads are employed. These are cross attention heads that map a sequence of input tokens onto a set of read-out tokens; the approach was popularized by the Perceiver-IO architecture [?] although it is essentially nothing else than the cross-attention in the original transformer decoder [?]. In particular, read-out heads are used to map from per dataset specific tokens to the global latent space and to provide predictions in physical space from the latent space.

More specifically, let $(X_i)_{i=1}^N$ be the set of input tokens and $(k_i^h)_{i=1}^N$ and $(v_i^h)_{i=1}^N$ be their projection onto the h^{th} attention head. Similarly, let $(q_i^h)_{i=1}^M$ be a set of head-specific query tokens, with q_i being either learned or fixed but notably not dependent on the model input. The output tokens $\bar{q}_j^h$ are then defined by cross attention, i.e.

$$\bar{q}_j^h = \sum_{i=1}^N \sigma(q_j^h \cdot k_i^h) v_i^h \quad (1)$$

where σ is the softmax function. The output of the multi-head attention module is, as usual, the concatenation of the per-head outputs $\bar{q}_j^h$ with a subsequent linear layer L applied to it, i.e.

$$\bar{X}_j = L(\text{cat}(\bar{q}_j^1, \dots, \bar{q}_j^H)). \quad (2)$$

8 Next steps

Two crucial aspects are missing in the current structure of the WeatherGenerator. One is a multi-scale structure in the in- and output. The second is to implement a multi-scale conditional latent diffusion model as the forecasting engine. See Fig. 5 for an overview.

9 The dirty details

This section collects a more detailed view of the model architecture and the data flow through it.

The input to the dataloader from the dataset classes is currently

10 Details on data flow in the code

Here we will follow the flow of the ERA5 data through the network, specifying the dimensions of the tensors as a proxy for the functions and information flow. We are discussing the single input dataset case here.

Have a look at the forward function in the model class at this point. Also have a look at the diagram here in Fig 3, which gives an idea of what goes into which function etc...

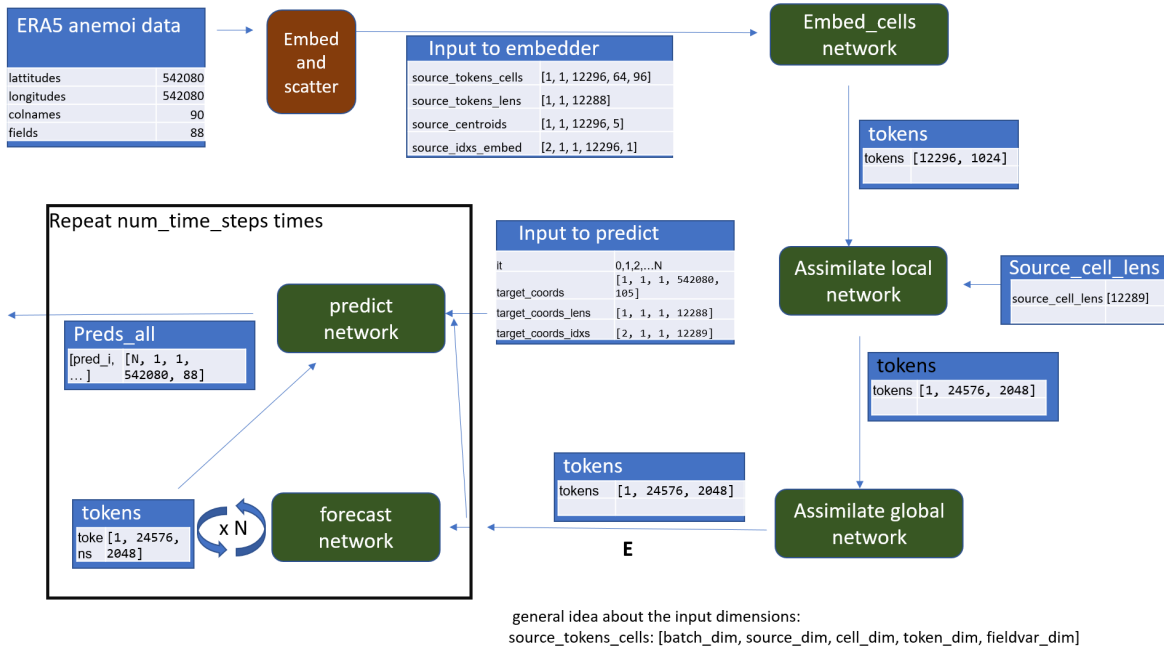


Figure 3: Diagram showing how data is flowing through the network

10.1 Input to the network

The ERA5 data has around 0.4° resolution in latitudes and 0.28° resolution in the longitudes, resulting in a total 542080 points on the lat/lon grid. Each of these 88 fields (temp, humidity etc) + 2 lat/lon coordinates, resulting in 90 columns in the zarr file.

10.2 Initial Embed and scatter network (self.embed_cells(model_params, source_tokens_cells, source_tokens_lens, source_centroids, source_idx embed))

First layer in the model is the embed cells, which consist of two main functionalities:

1. an embedding layer
 - (a) this can be a transofrmer or a linear layer or something else
2. and the scatter function and the scatter function

10.2.1 Input dimensions

The general dimensionality is like this: `source_tokens_cells`: [batch_dim, source_dim, cell_dim, token_dim, fieldvar_dim], example: [1, 1, 12296, 64, 96].

The `cell_dim` is in the simplest form how many healpix cells (areas) there are. For example for `healpix_level=5`, there are 12288 cells. `token_dim` is set as a parameter, that gives the maximum number of tokens per cell. So, let's say, if in a healpix cell, there are measurements on only 20 ERA5 grid points, then these twenty are simply appended by 44 0s to complete to the `token_dim` parameter of 64. If, on the other hand, there are more ERA5 grid points in the cell, say in total 76 measurement points, then a new cell is created to catch the "spillage". These first 64 of the measurements fill the first cell, the rest 12 spill over to the new created cell and get appended by 52 0's to fill to 64. The variable `source_idxes_embed` keeps track of the spillage.

So we can say we have 12296 buckets that hold a max of 64 tokens inside, for each cell, when a bucket is not full, it gets filled with 0s; when it's full, we put the spillage into a new bucket. So we can have more buckets than cells.

Remark: Analysis shows that for bucket size of 64 and ERA5 data, we have 41 tokens per healpix cell in average. So in average in each bucket 41 are coming from ERA5, 23 are filled with zeroes. If we chose a bucket size of 16, then in average we would have 3 buckets per cell, since $\text{ceil}(41/16) = 3$.

Note: Kerem proposes to use the word bucket instead of cell after the tokens are distributed to the healpix cells, because after this, we have multiple buckets ("cells") that are associated with the same healpix cell etc, so for me calling both "cell" is confusing for me.

10.2.2 Code

The first loop for `ib, sb` in `enumerate(source_tokens_cells)` is over the batch dimension. The second loop for `itype, (s, embed)` in `enumerate(zip(sb, self.embeds))` is over the source dimension.

Each source goes through the embedding network given in `self.embeds`, these are different for each source. The embedding is done per cell.

Then the function `tokens_all.scatter_(0, idxs, x_embed + model_params.pe_embed[idxs_pe])` puts all tokens together (from different batches and sources).

In the end, the `tokens_all` have dimensions [12296, 1024], with 1024 the `hidden_dim`. If we had two data streams (era5: `source_tokens_cells[0][0] => [12296, 64, 96]` and cosmo2: `source_tokens_cells[0][0] => [3572, 64, 96]`), then `tokens_all` would be [15868, 1024].

So the output has the shape [12296, 1024] for ERA5.

10.3 Local assimilation engine (`tokens = self.assimilate_local(model_params, tokens, source_cell_lens)`)

10.3.1 Inputs

This is still processing the data per cell.

The inputs are the tokens from before ([12296, 1024]) and the `source_cell_lens`, which is of size ([12289]). It looks like this [0, 1, 1, 1, 2, 1, 120, 1, ..., 8, 1, 1, ...], with most entries 1 and some >1. The first entry is set to 0 for technical purposes (making the length 12288+1=12289). Say the max entry in this vector is 131. This vector holds the information, how many token buckets (each with 64 tokens) there are in each cell. In the example here, we have the first three cells with one bucket each, then a cell with two buckets, then one cell with one bucket, then one cell with 120 buckets etc... We have one cell with 131 buckets.

As a sanity check: the sum of this vector `source_cell_lens` has to give us 12296. Or if we had two data sources (era5: `source_tokens_cells[0][0] => [12296, 64, 96]` and cosmo2: `source_tokens_cells[0][0] => [3572, 64, 96]`), then the sum of this vector would be 15868.

10.3.2 The code

The function implements a varlen attention mechanism. As we can have different number of tokens per cell (i.e. diff number of buckets) we need the varlen attention mechanism. It also uses the Perception IO idea to make the number of tokens independent of the number of input tokens. That is, the input may contain 2, 300, 25 or whatever many tokens per cell, the output will always be a fixed number of tokens. Concretely, in our example, we have 12288 cells. In our config we state how many tokens per cell we want (cf.local_num_queries), say local_num_queries=2, then we state we want 2 tokens per healpix cell regardless of how many tokens we have associated with that cell in the input. In this example we will end up with $12288*2=24576$ tokens in total.

The fixed output number of tokens is implemented with the perceiver IO idea, that is the queries come from randomly initialized vectors instead of being formed from the input tokens. In the code the query vectors are given in q_cells.

Remark: imagine we had set num_tokens (bucket size) to 8 instead of 64. As stated before, with ERA5 we have in average 41 tokens per cell, that would mean we would have $\text{ceil}(41/8)=6$ buckets associated with each healpix cell in average. so i would have an input size similar to [73728, 1024] ($73728=12288*6$). Then if i have local_num_queries=2, that is my output number of tokens is set to $12288*2=24576$, that would lead to a token compression $73728 \rightarrow 24576$, that is a ratio of $73728/24576=3$. So here one needs to be careful about this compression. Also be aware that this compression ration is not the same for all cells, as each cell has a different number of tokens/buckets...

Output: the code outputs the tokens in the size of [cf.local_num_queries*num_healpix_cells, 2048], in this example [24576, 2048].

Question: 2048 why is this 2048 and not 1024?

10.4 Global assimilation engine (tokens = self.assimilate_global(model_params, tokens))

10.4.1 The Input

This takes the tokens as inputs of size [24576,2048]. The idea here is to mix the information from the different healpix cells together for the first time. So it uses global as well as local attention blocks, the ratio is given by the parameter cf.ae_global_att_dense_rate, say if this is set to 0.2, then every 5th block is global attention.

10.4.2 The Code

The global attention layers use the full attention score matrix (24576×24576), although implemented with flex_attn in the background without forming the full matrix.

The local attention matrices are implemented by providing a mask. The mask filters for neighboring healpix cells, taking only the closest n'th neighbors. Finding out which cells are neighbors is very easy in the healpix algorithm. In the healpix algorithm, if two cells belong to the same mother healpix cell, their indices have the same quotient when divided by 4. Simply, cells 1,2,3,4 belong to the same mother cell, 5,6,7,8 belong to another mother cell.

Take, for instance cells number 154 and 149 in the Fig 4 and say we want to find the mother cells two layers above. so we need to take floor division with $4^2 = 16$. $\text{floor}(154/16)=9$, $\text{floor}(149/16)=9$, so both of these belong to the same mother two levels above. If we want to go three levels above, we need floor division with $4^3 = 64$.

This masking is implemented in the code as where block_factor=64 in our example. Have a look at the mask_block_local(batch, head, idx_q, idx_kv) function in the code at this point.

An immediate idea here is to experiment with varying block factors for different heads, that is, different heads looking at neighbors at different healpix levels.

10.4.3 Output

the global assimilation network mixes info together from the different cells and outputs tokens of size [1, 24576, 2048].

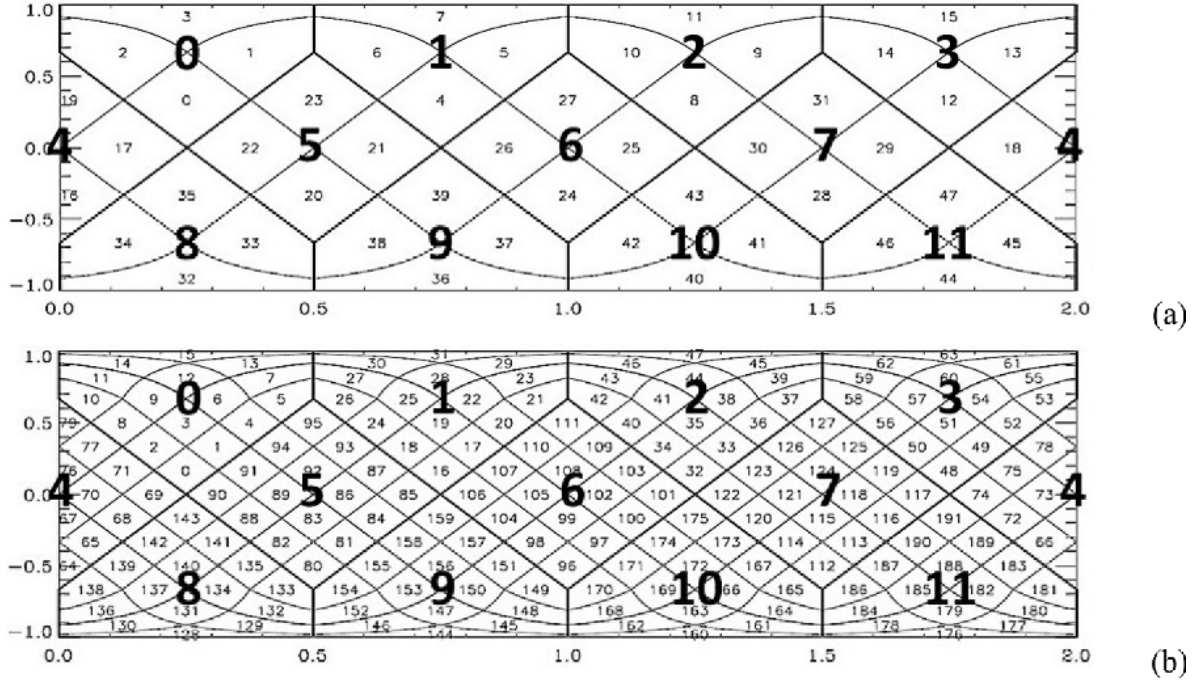


Figure 4: Healpix

target_coords	[1, 1, 1, 542080, 105]	542080: number of lat/lon grids, 105: coordinates in different coordinate
target_coords_lens	[1, 1, 1, 12288]	again tells us how many buckets per cell, sth like [41, 41, 42, ..., 41, 70, 41]
target_coords_idx	[2, 1, 1, 12289]	Question: what is the logic here?

10.5 Predict network (self.predict(model_params, it, tokens, target_coords, target_coords_lens, target_coords_idx))

this is called in an iterative manner in the rollout loop, 0 to num_time_steps. this also uses varlen attention, implementing the perceiver IO to go from a fixed number of tokens to variable output sizes.

10.5.1 Input

Inputs are the tokens from the global engine and the prescribed output coordinates.

remark: if we have two stream, era5 and cosmo2, then

target_coords[0][0][0]: Tensor shape=torch.Size([226980, 105])

target_coords[0][0][1]: Tensor shape=torch.Size([542080, 105]),

similarly

target_coords_lens[0][0][0]: Tensor shape=torch.Size([12288]), dtype=torch.int64, device=cpu tar-

get_coords_lens[0][0][1]: Tensor shape=torch.Size([12288]), dtype=torch.int64, device=cpu

10.6 Forecast network (tokens = self.forecast(model_params, tokens))

This takes the tokens as input, advances them and spits out the advanced tokens in the same format.

A Multi-resolution architecture

An overview of the envisioned multi-resolution structure is give in Fig. 5.

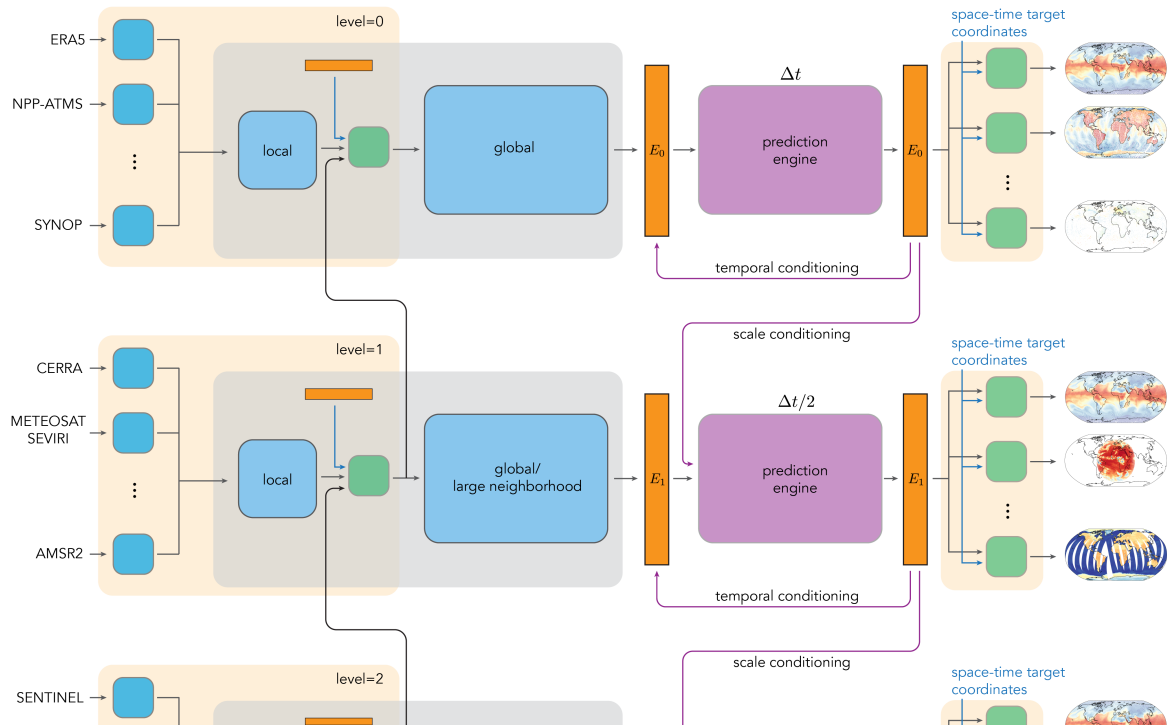


Figure 5: Overview of the WeatherGenerator neural network architecture